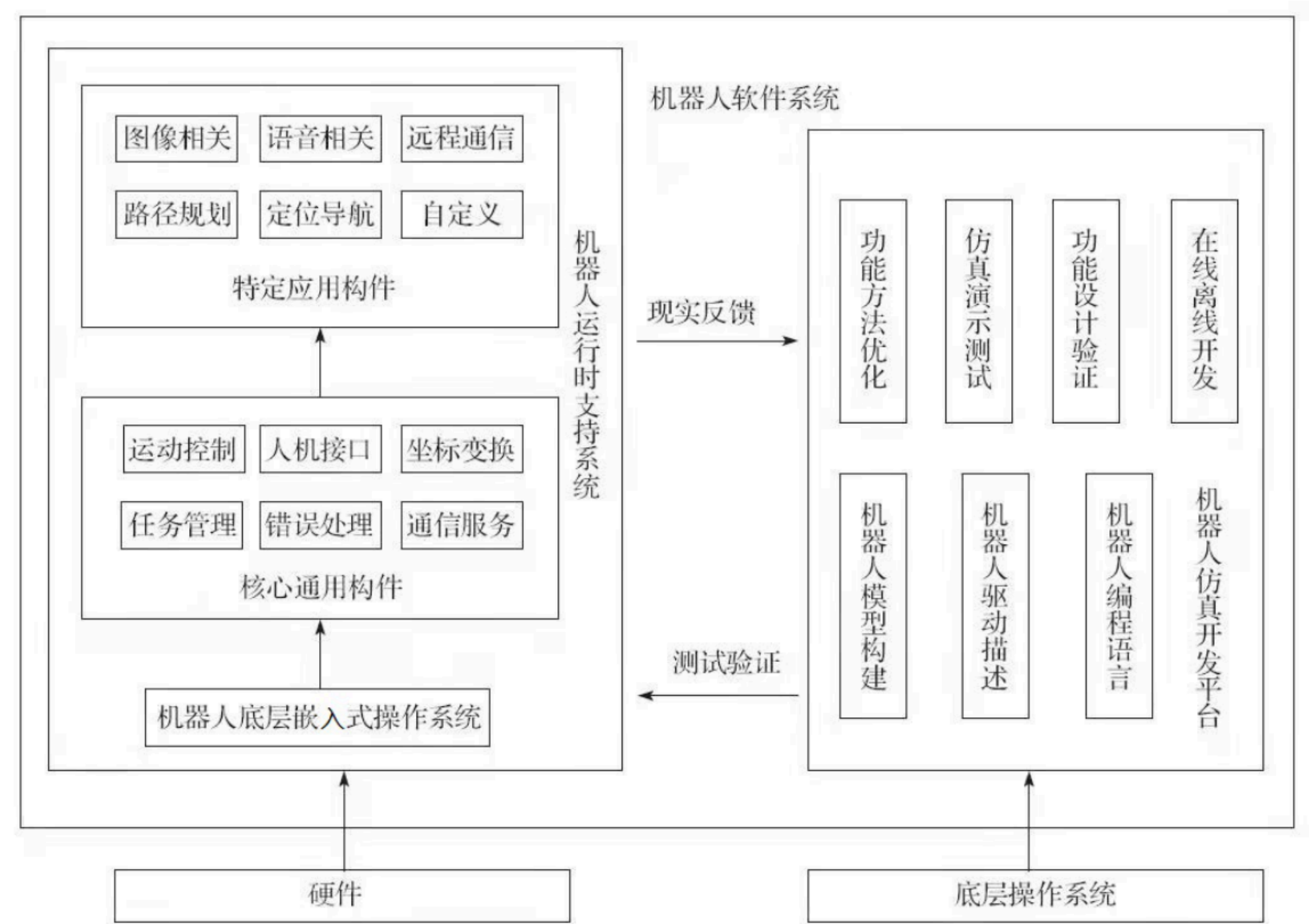


1 概述

机器人硬件组成

- 信息感知：智能机器人通过内部传感器实现自身状态感知，同时借助外部传感器获取环境信息
 - 内部传感器（自身感知）
 - 位置、位移、角度传感器
 - 速度、加速度传感器
 - 力觉传感器
 - 外部传感器（环境感知）
 - 视觉传感器
 - 触觉传感器
 - 声觉传感器
 - 滑觉传感器
- 决策控制：由控制计算机处理感知数据并生成行为指令
 - 通用计算机
 - 嵌入式计算机
 - 高性能单片机
- 驱动装置：包含能量转换装置及驱动电机，负责将控制信号转化为机械运动
- 机械结构：承载传感器、驱动与控制系统的实体框架，构成机器人物理形态

机器人软件系统



机器人软件构件化

构件化是提高机器人软件重用性和降低系统设计难度的主流方法。通过将机器人系统从功能上分解为各种可重用的构件，并建立构件的静态模型和运行时模型，不同构建之间的通信采用相应的中间件来实现，最终通过构件之间的整合高效的完成机

器人软件系统。

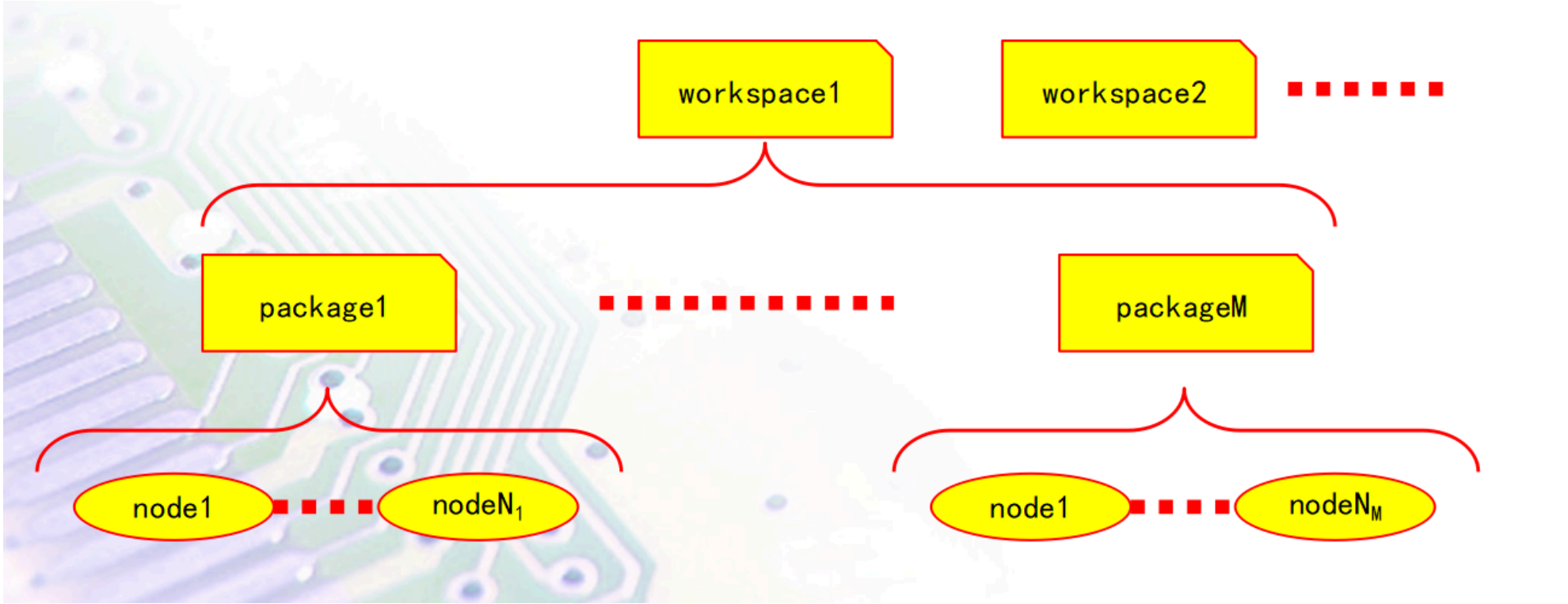
| ROS

ROS操作系统和传统操作系统有什么相似之处？

ROS操作系统能够提供类似传统操作系统的诸多功能，如硬件抽象、底层设备控制、进程间消息传递和程序包管理等。此外，它还提供相关工具和库，用于获得、编译、编辑代码以及在多个计算机之间运行程序完成分布式计算的功能。

| 2 功能包

| ROS2的层次架构



- **工作空间**：是开发ROS项目的顶层目录，包含所有功能包、编译文件和安装文件。
- **功能包**：是工作空间中的基本功能单元，对应一个特定功能的文件夹)。一个工作空间可以包含多个功能包。
- **节点**：是功能包中可执行的进程。一个功能包可以包含多个节点，每个节点负责完成一个具体的任务。

| 工作空间的结构

- src/：源代码空间。用户在这里创建和存放所有功能包的源代码。
- build/：编译空间。colcon工具存储缓存信息、配置参数和中间编译文件的地方。
- install/：安装空间。构建成功后，生成的可执行文件、库文件和脚本都会安装到此目录。
- log/：日志空间。存储构建过程中的日志信息。

常用命令：

```
新建工作空间目录
mkdir -p ./ros_workspace/src

构建功能包
colcon build
构建过程中，当前目录下会产生build、install和log目录
符号链接构建（用路径符号代替实体文件）：
colcon build --symlink-install

只构建指定包：
colcon build --packages-select <package_name>
构建指定包及其相关包：
colcon build --packages-up-to <package_name>
```

| 功能包

- python功能包：以py文件实现节点功能
- c++功能包：以cpp文件实现节点功能

|python功能包

- 创建功能包

```
ros2 pkg create --build-type ament_python --license Apache-2.0 <package_name>
```

- 创建并编写py模块代码

```
touch hello.py
```

编写代码

- 设置setup.py

添加终端脚本(console_scripts):

```
'hello_cr = py_hello_ros.hello:main'
```

hello_cr为ros2指令中的run指令脚本名，py_hello_ros,hello,main分别是功能包名、模块名、函数名

- 构建功能包

```
colcon build
```

- 运行程序

```
ros2 run py_hello_ros hello_cr
```

- 可能遇到环境变量设置问题

🔗 环境变量设置问题

1. 临时处理

```
source ~/Desktop/ros_workspace/install/setup.bash
```

2. 永久设置

```
echo "source ~/Desktop/ros_workspace/install/setup.bash" >> ~/.bashrc
```

|C++功能包

- 创建功能包

```
ros2 pkg create --build-type ament_cmake --license Apache-2.0 <package_name>
```

- 创建并编写C++代码

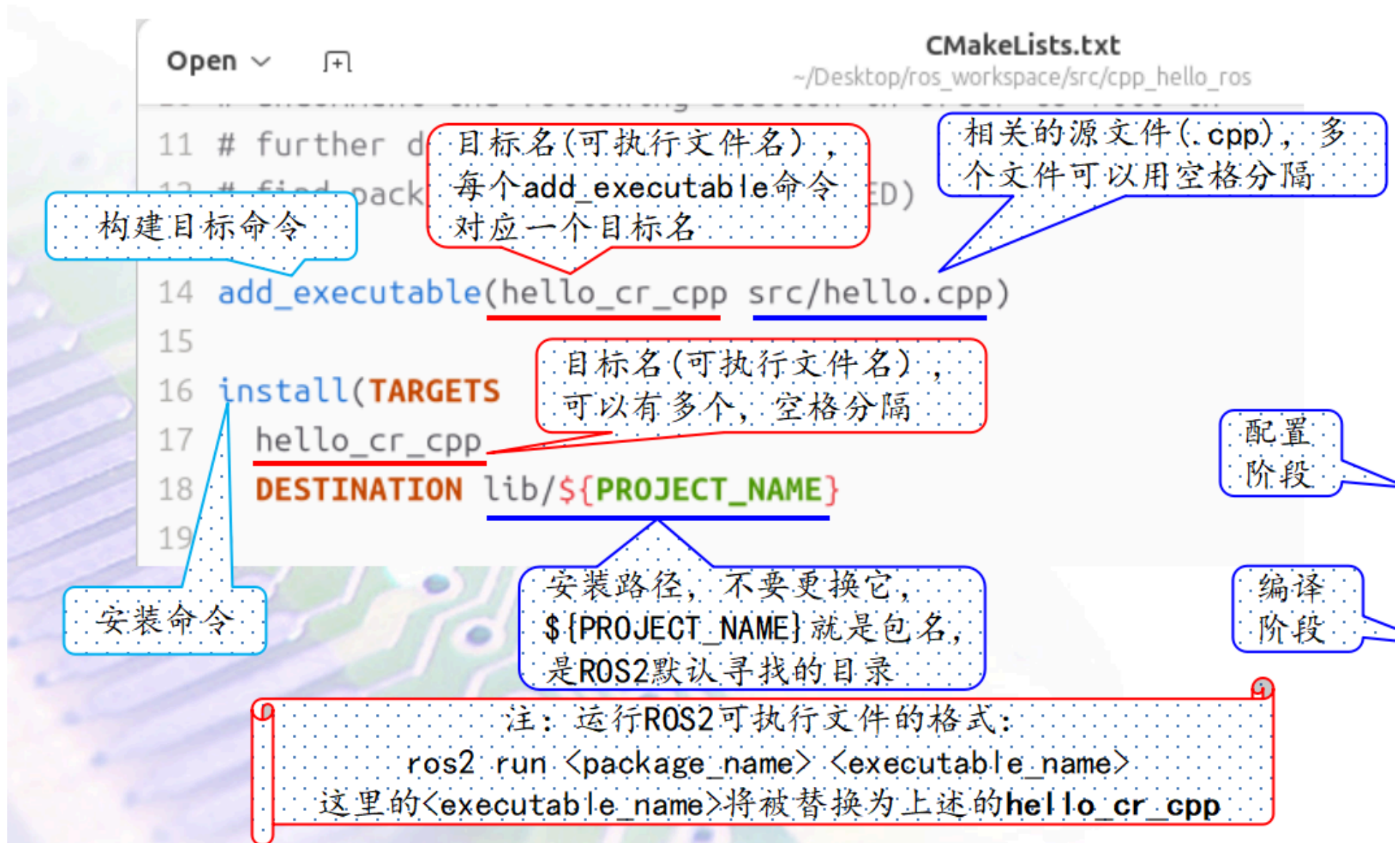
```
touch hello.cpp
```

编写代码

- 设置CMakeLists.txt

◆ 设置CMakeLists.txt

✎ 添加构建目标和安装命令



- colcon build
- ros2 run

| 作业

在ROS2系统中创建工作空间并生成一个python功能包"py_yanghui_triangle", 请编写python模块文件, 要求在终端运行"ros2 run py_yanghui_triangle yanghui"后, 能够输出4-10行的杨辉三角形。请给出具体的操作步骤。

- 创建工作空间目录

```
mkdir -p ~/Desktop/my_ws/src
```
- 在src目录中执行创建功能包

```
ros2 pkg create --build-type ament_python --license Apache-2.0 py_yanghui_triangle
```
- 在生成的"py_yanghui_triangle/py_yanghui_triangle"目录中创建yanghui_triangle.py文件(文件名自定), 并编写程序:

```

N=10
L=[[1]]

def print_list_format():
    for i in range(0, len(L)):
        print(' '*4*(len(L)-i-1), end='')
        for k in range(0, len(L[i])):
            print(f'{L[i][k]:^8}', end='')
        print()

def main():
    for i in range(1, N):
        list_temp=[1]
        for j in range(i):
            try:
                list_temp.append(L[i-1][j]+L[i-1][j+1])
            except IndexError:

```

```
list_temp.append(L[i-1][j]+0)
L.append(list_temp)
print_list_format()
```

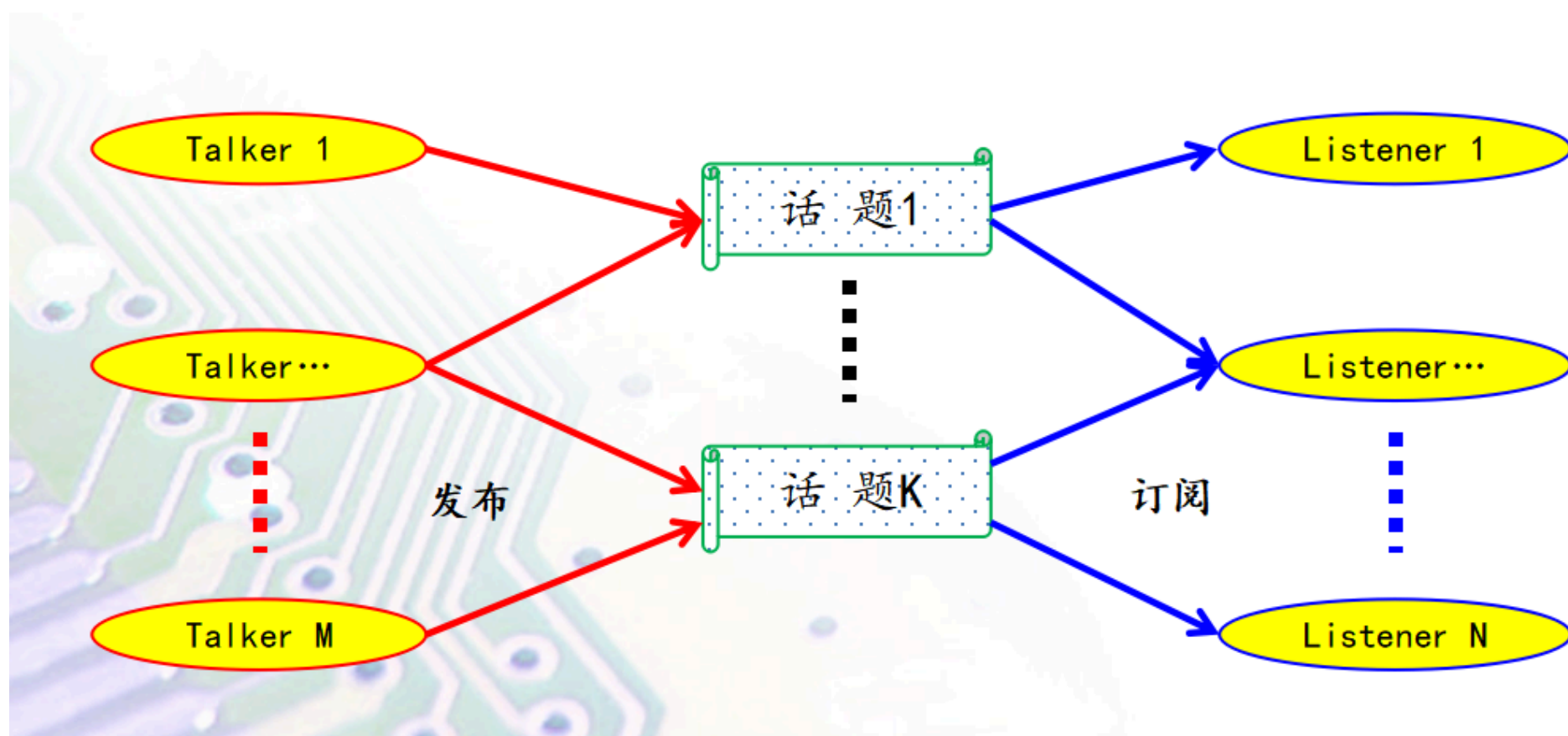
- 编辑setup.py文件
`yanghui = py_yanghui_triangle.yanghui_triangle:main`
- （如未设置）添加环境变量-source命令
- 构建功能包 `colcon build`
- 运行 `ros2 run py_yanghui_triangle yanghui`

3 节点通信

节点通信的4种方式：

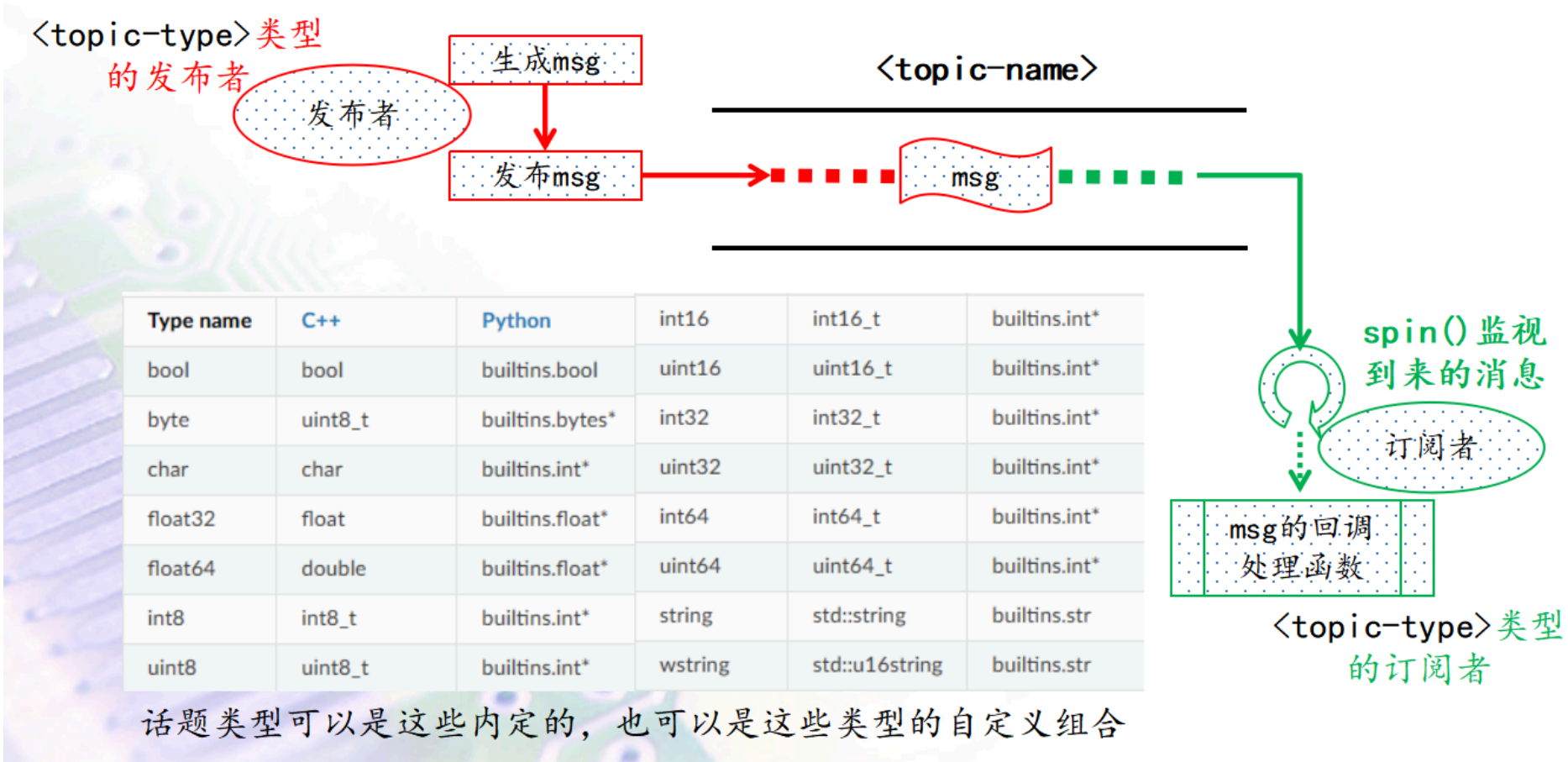
- 话题通信(发布订阅模式)
- 服务通信(请求响应模式)
- 动作通信(请求响应+连续反馈模式)
- 参数服务

话题通信



特点：

- 解耦性好
 - 用于不断更新，计算处理少的场景
- 流程：



话题通信编程

- 创建功能包

```
ros2 pkg create --build-type ament_python --license Apache-2.0 py_demo_topic --dependencies rclpy std_msgs
```

--dependencies中是依赖包的包名，空格分隔

- 编写发布者节点

```
# 导入ROS2客户端库
import rclpy
# 导入Node基类，所有节点都需要继承此类
from rclpy.node import Node
# 导入标准消息类型String
from std_msgs.msg import String

class DemoPublisher(Node):
    """发布者节点类，用于定期发布消息到指定话题"""

    def __init__(self):
        """
        节点初始化函数
        - 创建节点名为'my_publisher'
        - 创建发布者对象，话题名为'chd_topic'，队列大小为10
        - 创建定时器，每0.5秒触发一次回调函数
        """
        # 调用父类构造函数，设置节点名
        super().__init__('my_publisher')

        # 创建发布者对象：
        # - String：消息类型（标准字符串消息）
        # - 'chd_topic'：话题名称（其他节点通过此名称订阅）
        # - 10：队列大小（缓存的消息数量）
        self.publisher_chd = self.create_publisher(String, 'chd_topic', 10)

        # 设置定时器周期为0.5秒
        timer_period = 0.5 # seconds

        # 创建定时器：
        # - timer_period：定时器周期（秒）
        # - self.timer_callback：定时器到期时的回调函数
        self.timer_chd = self.create_timer(timer_period, self.timer_callback)

        # 初始化消息索引计数器
```

```

self.index = 0

# 在日志中输出初始化完成信息
self.get_logger().info('DemoPublisher节点初始化完成，开始发布消息...')

def timer_callback(self):
    """
    定时器回调函数
    - 每0.5秒自动执行一次
    - 创建并发布消息到'chd_topic'话题
    - 在日志中记录发布的消息内容
    """
    # 创建String类型的消息对象
    msg = String()

    # 设置消息内容：包含问候语和当前索引
    msg.data = f'Hello World and CHD: index={self.index}'

    # 发布消息到'chd_topic'话题
    self.publisher_chd.publish(msg)

    # 在日志中记录发布的消息内容（INFO级别）
    self.get_logger().info(f'Publishing: "{msg.data}"')

    # 索引自增，为下一条消息准备
    self.index += 1

def main(args=None):
    """
    主函数：节点的主要执行流程
    """
    # 初始化ROS2客户端库，必须首先调用
    rclpy.init(args=args)

    try:
        # 创建发布者节点实例
        demo_publisher = DemoPublisher()

        # 进入ROS2事件循环，保持节点运行并处理回调
        # spin()会阻塞直到节点被关闭
        rclpy.spin(demo_publisher)

    except KeyboardInterrupt:
        # 捕获Ctrl+C中断信号
        demo_publisher.get_logger().info('收到中断信号，节点正在关闭...')
    except Exception as e:
        # 捕获其他异常
        demo_publisher.get_logger().error(f'节点运行出错：{e}')
    finally:
        # 清理资源：销毁节点
        if 'demo_publisher' in locals():
            demo_publisher.destroy_node()

        # 关闭ROS2客户端库
        rclpy.shutdown()

# Python标准的主程序入口
if __name__ == '__main__':
    main()

```

- 编写订阅者节点


```
# 导入ROS2客户端库
import rclpy
# 导入Node基类，所有节点都需要继承此类
from rclpy.node import Node
# 导入标准消息类型String
from std_msgs.msg import String

class DemoSubscriber(Node):
    """订阅者节点类，用于订阅指定话题并接收消息"""

    def __init__(self):
        """
        节点初始化函数
        - 创建节点名为'my_subscriber'
        - 创建订阅者对象，订阅'chd_topic'话题
        - 设置消息接收时的回调函数
        """
        # 调用父类构造函数，设置节点名
        super().__init__('my_subscriber')

        # 创建订阅者对象：
        # - String： 订阅的消息类型（必须与发布者发送的类型匹配）
        # - 'chd_topic'： 要订阅的话题名称
        # - self.listener_callback： 消息到达时的回调函数
        # - 10： 队列大小（缓存的消息数量）
        self.subscription_chd = self.create_subscription(
            String,
            'chd_topic',
            self.listener_callback,
            10)

        # 这行代码是为了防止未使用变量警告（PEP8规范）
        # 在实际功能上不是必需的，但可以保持代码整洁
        self.subscription_chd # prevent unused variable warning

        # 在日志中输出初始化完成信息
        self.get_logger().info('DemoSubscriber节点初始化完成，开始监听chd_topic话题...')

    def listener_callback(self, msg):
        """
        消息接收回调函数
        - 当有消息发布到'chd_topic'话题时自动调用
        - msg： 接收到的消息对象

        :param msg: 接收到的String类型消息
        """
        # 在日志中记录接收到的消息内容
        # 使用f-string格式化输出，显示消息内容
        self.get_logger().info(f'I heard: "{msg.data}"')

def main(args=None):
    """
    主函数：节点的主要执行流程
    """
    # 初始化ROS2客户端库，必须首先调用
    rclpy.init(args=args)

    try:
        # 创建订阅者节点实例
        demo_subscriber = DemoSubscriber()

        # 在日志中输出节点启动信息
        demo_subscriber.get_logger().info('订阅者节点启动，进入消息监听状态...')
```



```

# 进入ROS2事件循环，保持节点运行并处理回调
# spin()会阻塞直到节点被关闭，期间会监听到来的消息
rclpy.spin(demo_subscriber)

except KeyboardInterrupt:
    # 捕获Ctrl+C中断信号
    demo_subscriber.get_logger().info('收到中断信号，节点正在关闭...')
except Exception as e:
    # 捕获其他异常
    demo_subscriber.get_logger().error(f'节点运行出错：{e}')
finally:
    # 清理资源：销毁节点
    # 修正：这里应该是demo_subscriber，不是minimal_subscriber
    if 'demo_subscriber' in locals():
        demo_subscriber.destroy_node()
        demo_subscriber.get_logger().info('节点资源已清理')

# 关闭ROS2客户端库
rclpy.shutdown()

# Python标准的主程序入口
if __name__ == '__main__':
    main()

```

- 配置安装文件setup.py
同之前
- 构建colcon build
- 运行ros2 run

◆ 程序要点回顾

☞ 程序主流程

- 初始化rclpy:

```
rclpy.init()
```

- 创建对象:

```
demo_publisher = DemoPublisher()
```

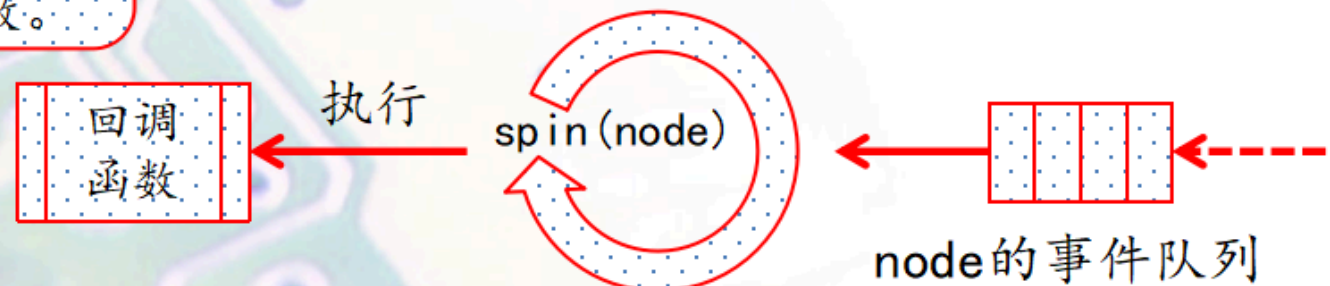
```
demo_subscriber = DemoSubscriber()
```

- spin循环等待（监控）:

spin(node) 监控node的订阅、定时器、服务、动作等回调，并在事件触发时调用对应的回调函数。

```
rclpy.spin(demo_publisher)
```

```
rclpy.spin(demo_subscriber)
```



☞ 节点类初始化(__init__)函数

- 初始化基类:

```
super().__init__('<节点名>')
```

- 创建Publisher/Subscription对象:

```
create_publisher(...)
```

```
create_timer(...)
```

```
create_subscription(...)
```

☞ 回调函数



自定义话题通信

需求：自定义发送消息的格式，如多个变量，多种数据类型
要使用C++制作接口功能包

- 创建接口功能包

```
ros2 pkg create --build-type ament_cmake --license Apache-2.0 gps_msg
```


- 新建msg目录和gps.msg文件并自定义消息类型

`mkdir msg && touch msg/gps.msg`

在gps.msg文件中，添加消息数据类型模板

```
string state
float32 x
float32 y
```

- 为package.xml添加依赖
- 编辑CMakeLists.txt

◆ 编辑CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(gps_msg)

find_package(std_msgs REQUIRED)
find_package(rosidl_default_generators REQUIRED)

set(msg_files
  "msg/Gps.msg"
)

rosidl_generate_interfaces(${PROJECT_NAME}
  ${msg_files}
  DEPENDENCIES std_msgs
)
ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

定义消息文件路径
变量msg_files

设置消息文件路径
到rosidl

使任何依赖gps_msg功能包的其
它功能包会自动获得
rosidl_default_runtime依赖，
简化依赖管理。

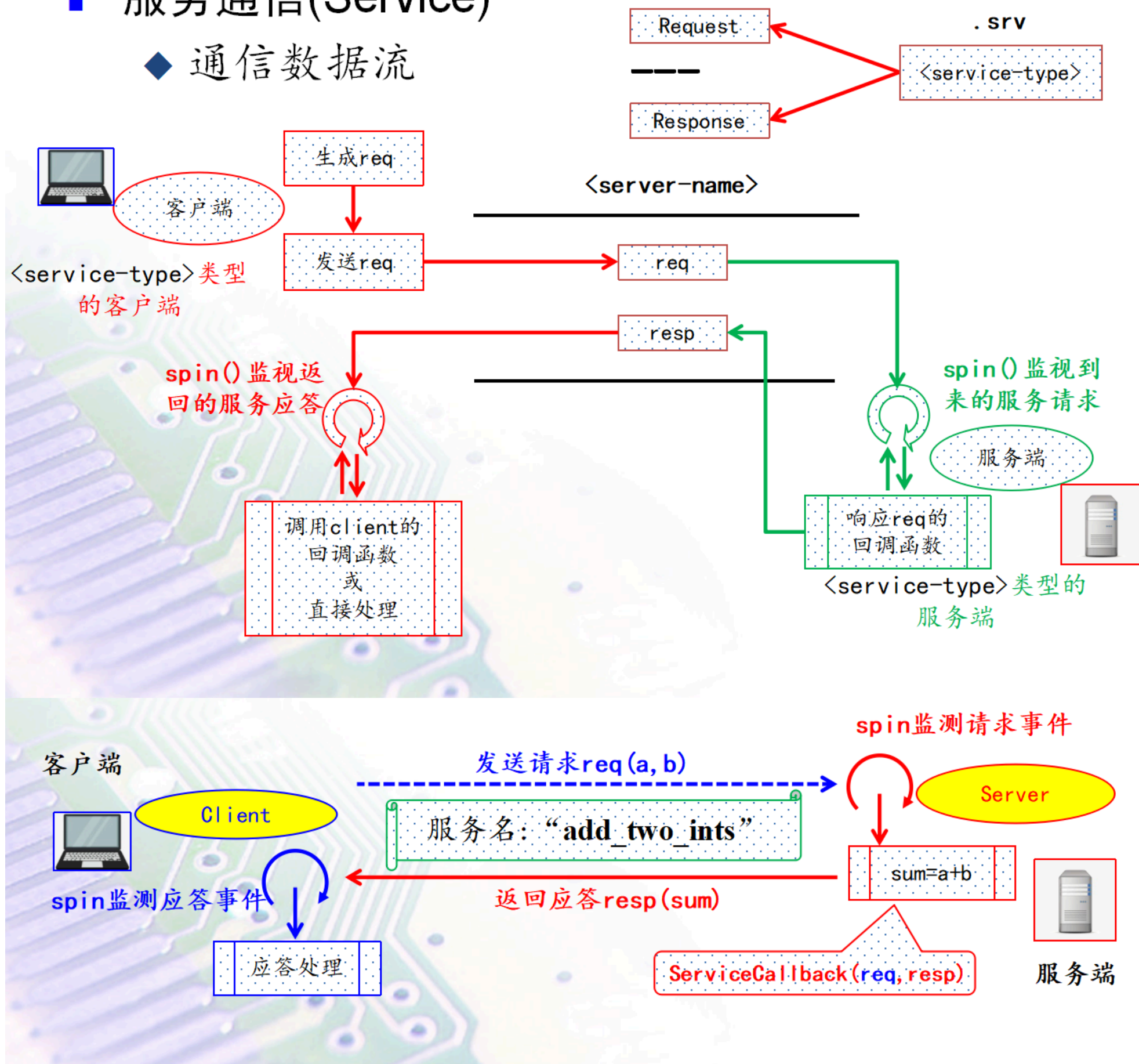
- 构建功能包colcon build
- 创建应用功能包


```
ros2 pkg create --build-type ament_python --license Apache-2.0 py_gps_topic
```
- package.xml中添加依赖信息
- 新建并编辑模块py文件
- 设置setup.py文件
- 构建功能包colcon build
- 运行ros2 run

服务通信

■ 服务通信(Service)

◆ 通信数据流



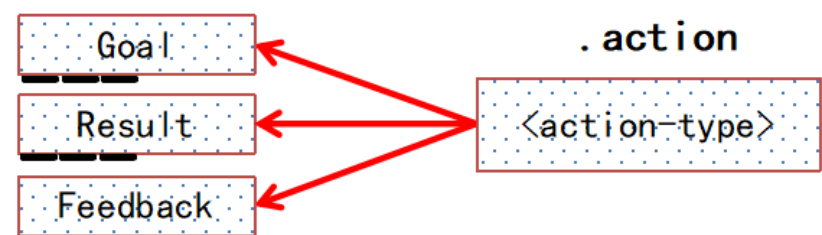
服务通信编程

- 创建功能包
`ros2 pkg create --build-type ament_python --license Apache-2.0 py_add_service`
- package.xml中添加依赖
`<depend>rclpy</depend>`
`<depend>chd_interfaces</depend>`
- 编写服务端节点
- 编写客户端节点
- 配置安装文件setup.py
- 构建colcon build
- 运行ros2 run

动作通信

■ 动作通信(Action)

◆ 通信数据流



<action-type> 动作 (action)

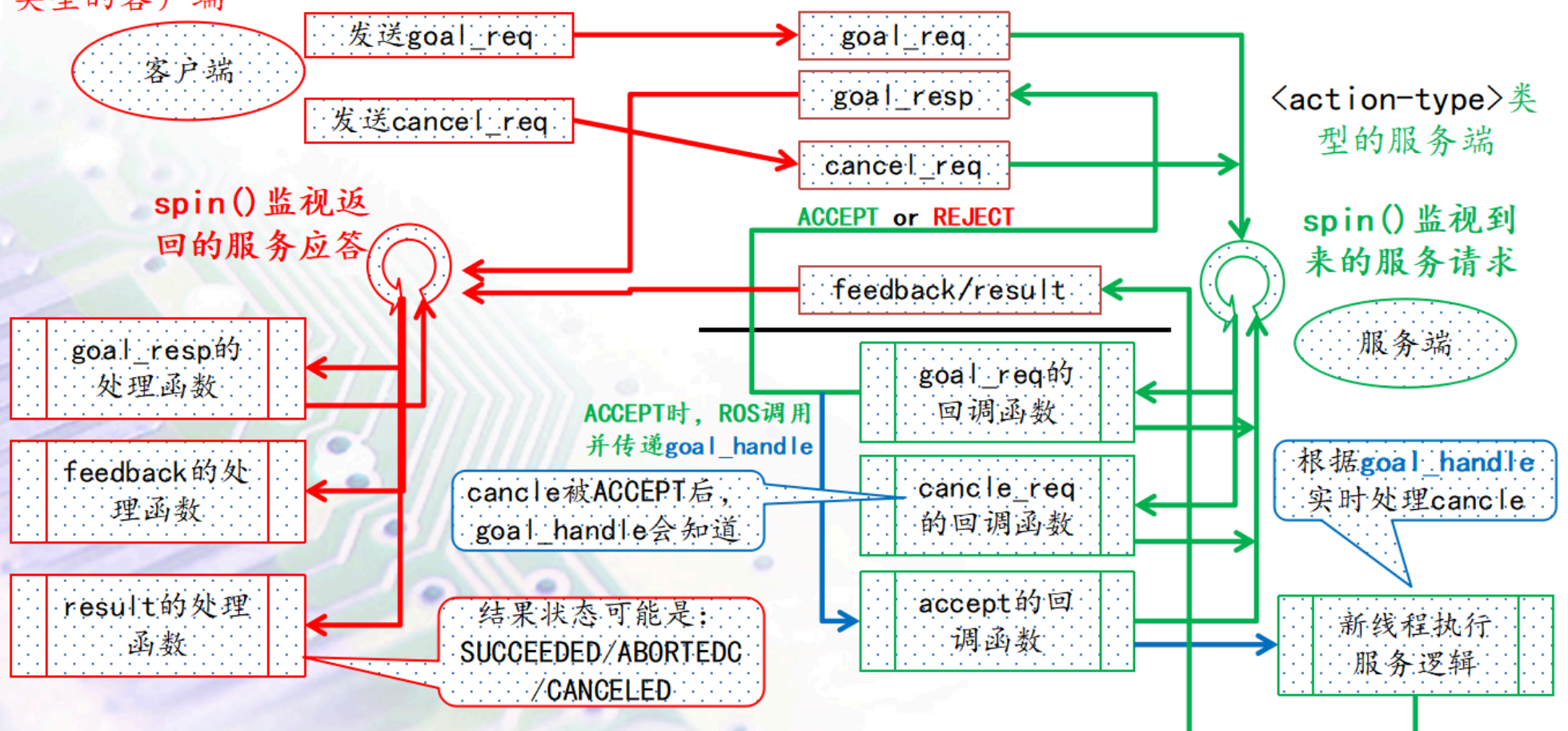
类型的客户端

<action-name>

<action-type> 类型的服务端

spin() 监视到来的服务请求

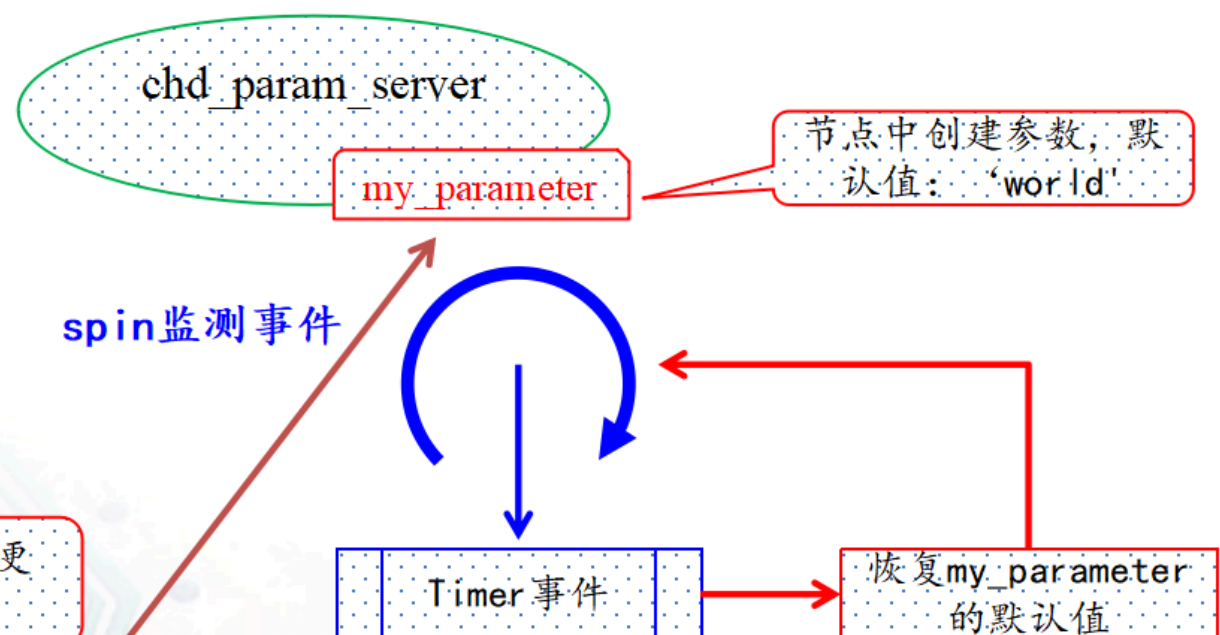
spin() 监视返回的服务应答



■ 动作通信编程

- 创建功能包
- package.xml添加依赖
- 编写服务端、客户端节点
- 编写setup.py
- 构建
- 运行

■ 参数服务



其他节点或指令更改节点的参数

```
hlcheng@chdubuntu:~$ ros2 param set /chd_param_server my_parameter chd
```

参数服务编程

- 创建功能包
- package.xml添加依赖
- 编写服务端、客户端节点
- 编写setup.py
- 构建
- 运行

节点的命名空间、重命名

命名空间namespace

将命名分组储存，便于管理，允许不同命名空间中存在重名。

默认的命名空间是根目录，即 '/'

命名空间路径的重映射方法：

```
ros2 run <package-name> <executable> --ros-args --remap __ns:=<new-ns>
```

<executable>填要被重命名的空间名

__ns代表这里重命名的是namespace，下面的__node等同理

节点重命名

```
ros2 run <package-name> <executable> --ros-args --remap __node:=<new-node-name>
```

如果要同时重命名命名空间和节点：

```
ros2 run <package-name> <executable> --ros-args --remap __ns:=<new-ns>--remap __node:=<new-node-name>
```

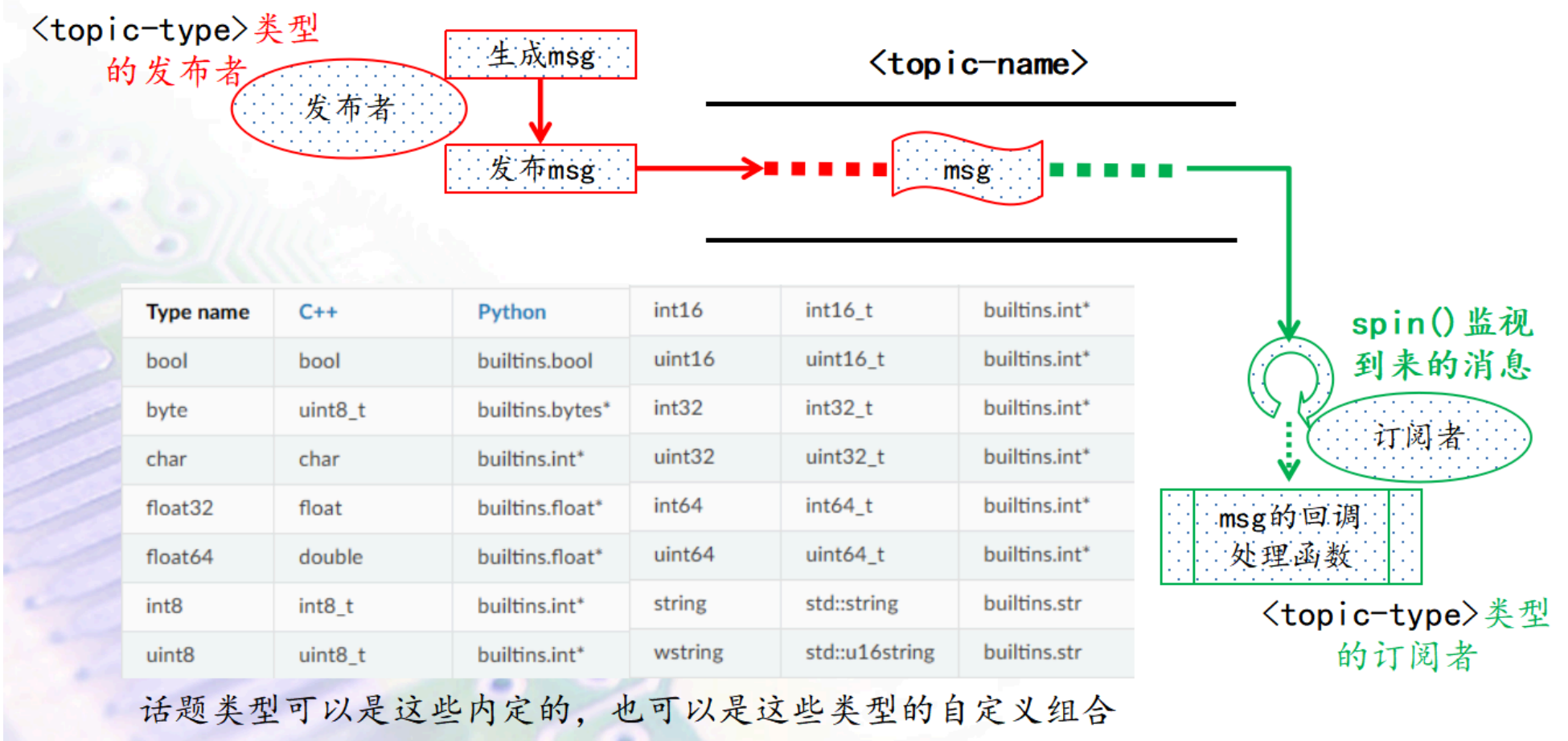
话题重命名

```
ros2 run <package-name> <executable> --ros-args --remap <old-topic-name>:=<new-topic-name>
```

作业

话题通信程序中，订阅者的回调函数作用是什么？该函数和spin函数有什么关系？请描述大概的处理过程。

订阅者的回调函数用于处理接收到的话题消息。spin函数是一个阻塞函数，不断轮询到达节点的事件，一旦发现有所订阅的消息到来就会通过线程去调用回调函数，实现消息的处理。



为了通过话题通信方式将机器人的位置和环境温湿度信息发布到系统中（位置用x、y、z表示，温度用t表示、湿度用h表示），要求用自定义接口实现该话题类型，应该如何定义该接口文件？在python模块中如何导入该类型以便使用？

可以创建一个C++功能包my_interface实现接口类型。比如在msg文件夹中创建Condition.msg文件，内容包括：

- float 64 x
- float 64 y
- float 64 z
- float 32 t
- float 32 h

通过构建该功能包，生成对应的C++和python接口类型Condition

在python模块中可使用from my_interface.msg

import Condition即可导入该类型，类型名为Condition(参考上面正文的自定义话题通信)

| 为了只针对特定功能包进行构建，可以使用哪些构建选项，这些选项有什么区别？

可采用构建选项：--packages-select或--packages-up-to进行构建，前者只构建指定包，不会处理其依赖的包；后者会递归构建所有依赖，将所有相关的功能包同步更新。

如 `colcon build --packages-select <package_name>`

| 构建选项中的一symlink-install的作用是什么？

该选项能够在安装目录通过符号链接的方式指向二进制文件和资源文件，减少了默认没有该选项产生的文件I/O操作，更重要的是符号链接直接指向源码文件，更改python源码和资源文件无需重新构建，提升了调试效率。

| 4 Launch

| 意义

- 无须手动多次进行ros2 run，一键启动要运行的东西
`ros2 launch [<package-name>] <launch-file-name>`
- 可设置启动条件、设置节点参数、设置环境变量
- 可包含其他 launch 脚本
- 应位于src/<package>/launch中

会涉及到.py .xml .yaml格式文件

| 基本格式

- 导入LaunchDescription类

```
from launch import LaunchDescription
```

- 进行全局常量、函数、工具定义
- 定义回调函数（就是launch启动过程中跑的函数）

```
def generate_launch_description( ):
```

- 在回调函数中，写所有需要进行的动作，如启动节点（使用Node类）等，这些动作称为actions
- 返回LaunchDescription对象，括号里面装刚才定义的 actions

```
return LaunchDescription([ ... ])
```


在此过程中可能用到的函数：

□ 常用函数

后续的msg、srv、urdf、launch等重要文件夹的存储位置

□ 获取功能包的安装路径

```
from ament_index_python.packages import get_package_share_directory
package_path = get_package_share_directory('<package-name>')
```

□ 路径拼接

```
import os
<path-str>=os.path.join('<folder-name>','folder-name',...)
```

□ 获得所匹配文件的列表

```
from glob import glob
glob('launch/*.py')→返回所有.py文件列表
```

| 具体使用与编程

| 使用launch文件启动Node节点

- 导入所需的类

```
from launch import LaunchDescription
from launch_ros.actions import Node
```

- 在回调函数generate_launch_description()中生成node实例

```
def generate_launch_description( ):
    <node-ins>=Node(<初始化列表参数>)
```

列表参数要有：

- package='<package-name>' 功能包名
- executable='<executable-name>' 可执行文件名
- name='node-name' 节点名（可选）
- parameters=[{<param-dic>},{<param-dic>},...] 参数列表（可选）
- arguments=['<arg1>','<arg2>',...] 命令行参数（可选）
- output='log'（默认）或'screen'或'both' 输出控制（可选）

- return 返回，结束函数

```
return LaunchDescription([
    <node-ins>,
    ...,
])
```

- 设置setup.py

```
data_files=[
    (<目标文件夹路径>,[<待复制文件路径>,<...>]),
```

```
(...),  
]
```

- 构建，运行

使用launch文件运行 shell 命令

同样是在回调函数generate_launch_description()中编写。

格式：

launch文件最开头需要引入：

```
from launch.actions import ExecuteProcess
```

回调函数内：

```
<cmd-ins>= ExecuteProcess(  
    cmd = ['<cmd-str>',  
           '<cmd-str>',  
           ...  
          ],  
    output = 'screen' or 'log' or 'both',  
    shell = True or False
```

cmd-str是要输入运行的字符串，shell是决定是否按 shell 特性运行

例程：

```
from launch import LaunchDescription  
from launch.actions import ExecuteProcess  
  
def generate_launch_description():  
    cmd1_action = ExecuteProcess(  
        cmd = [  
            'echo',  
            '$AMENT_PREFIX_PATH',  
        ],  
        output = 'screen', shell = True  
    )  
    cmd2_action = ExecuteProcess(  
        cmd = [  
            'ros2',  
            'run',  
            'py_yanghui_triangle', 'yanghui'  
        ],  
        output = 'screen', shell = True  
    )  
    return LaunchDescription([  
        cmd1_action,  
        cmd2_action  
    ])
```

实现启动参数配置

要用到的两个关键的类：

DeclareLaunchArgument -定义参数

LaunchConfiguration -获取实际参数值

- 声明定义参数

```
from launch.actions import DeclareLaunchArgument  
  
# 声明一个背景色红色分量的参数  
decl_bg_r = DeclareLaunchArgument(  
    name='background_color_red',  
    default_value='0.0',  
    description='Background color red component',  
    allow_undefined=True
```

```

name='bg_r',          # 参数名称
default_value='255',  # 默认值（字符串类型）
description='Red color for background', # 参数描述
choices=['0', '128', '255'] # 可选值限制（可选）
)

```

- 获取参数值

```

from launch.substitutions import LaunchConfiguration

# 获取参数的值
red_value = LaunchConfiguration('bg_r') # 获取名为'bg_r'的参数值
green_value = LaunchConfiguration('bg_g')
blue_value = LaunchConfiguration('bg_b')

```

- 使用参数值

```

from launch_ros.actions import Node

# 在节点配置中使用参数
turtle_node = Node(
    package='turtlesim',
    executable='turtlesim_node',
    parameters=[{
        'background_r': red_value, # 使用参数值
        'background_g': green_value,
        'background_b': blue_value
    }]
)

```

| 实现文件包含

例如包含其他launch文件

两个需要用到的关键的类：

IncludeLaunchDescription - 包含动作

PythonLaunchDescriptionSource -launch文件源，将launch文件包装成标准格式

例程：

```

from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch import LaunchDescription
from ament_index_python import get_package_share_directory
import os

def generate_launch_description():
    # 1. 包含第一个Launch文件（机器狗控制）
    include_dog_action = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(
                get_package_share_directory('spotmicro_dog'), # 机器狗功能包
                'launch', # launch文件夹
                'spot_launch.py' # 机器狗的主Launch文件
            )
        )
    )

    # 2. 包含第二个Launch文件（乌龟模拟器），并传递参数
    include_turtle_action = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(

```



```

        os.path.join(
            get_package_share_directory('launch_study'), # 当前功能包
            'launch', # launch文件夹
            'param_launch.py' # 之前创建的参数化Launch文件
        )
    ),
    # 关键：向被包含的Launch文件传递参数
    launch_arguments=[
        ('bg_r', '100'), # 设置背景红色=100
        ('bg_g', '128'), # 设置背景绿色=128
        ('bg_b', '32') # 设置背景蓝色=32
    ]
)

# 3. 返回包含动作
return LaunchDescription([
    include_dog_action, # 启动机器狗系统
    include_turtle_action # 启动带特定参数的乌龟模拟器
])

```

I 实现分组设置

将节点的 action 分组，方便管理

所需的类：

GroupAction -分组动作

PushRosNamespace -命名空间推送,位组内的节点加上命名前缀

例程：

```

from launch import LaunchDescription
from launch_ros.actions import Node
from launch_ros.actions import PushRosNamespace
from launch.actions import GroupAction

def generate_launch_description():
    # 1. 创建三个独立的乌龟模拟器节点
    turtle1 = Node(
        package='turtlesim',
        executable='turtlesim_node',
        name='t1' # 原本会叫 /t1
    )

    turtle2 = Node(
        package='turtlesim',
        executable='turtlesim_node',
        name='t2' # 原本会叫 /t2
    )

    turtle3 = Node(
        package='turtlesim',
        executable='turtlesim_node',
        name='t3' # 原本会叫 /t3
    )

    # 2. 创建第一个分组：将turtle1和turtle2放入"Group_One"命名空间
    g1 = GroupAction([
        PushRosNamespace('Group_One'), # 设置命名空间前缀
        turtle1, # 现在全名： /Group_One/t1
        turtle2 # 现在全名： /Group_One/t2
    ])

    # 3. 创建第二个分组：将turtle3放入"Group_Two"命名空间

```

```
g2 = GroupAction([
    PushRosNamespace('Group_Two'), # 不同的命名空间
    turtle3 # 现在全名: /Group_Two/t3
])

# 4. 返回包含两个分组的Launch描述
return LaunchDescription([g1, g2])
```

| 事件启动设置

略

| 作业

功能包“chd_pkg”中有节点执行脚本“chd_execute”，该节点中有话题名为“/chd_topic”。请用命令行ros2 run的方式执行该脚本，要求：

- (1) 指定节点名为“new_name_node”；
- (2) 指定命名空间名为“/chd_ns”。
- (3) 将原话题名重映射为“/chd_new_topic”

该命令行应如何撰写？

如果用launch.py运行该节点，请编写该python文件（注，本题有自学内容，请学习重映射相关参数的用法）。

命令行：

```
ros2 run chd_pkg chd_execute --ros-args -r __node:=new_name_node -r __ns:=/chd_ns -r /chd_ns/chd_topic:=/chd_new_topic
```

launch.py：

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    the_node = Node(
        package = 'chd_pkg',
        executable = 'chd_execute',
        name = 'new_name_node',
        namespace= 'chd_ns',
        remappings=[
            ('chd_topic', '/chd_new_topic'),
        ],
    )

    return LaunchDescription([
        the_node
    ])
```

请撰写“chd_launch.py”文件内容，包含配置参数“my_arg”，并通过ExecuteProcess执行echo命令打印该参数的结果。要求该参数默认值为“1234”，通过命令行传入参数为“abcd”，请在代码后贴执行结果图。

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, ExecuteProcess
from launch.substitutions import LaunchConfiguration

def generate_launch_description():
    launch_conf=LaunchConfiguration('my_arg')
    decl_my_arg=DeclareLaunchArgument(
        name='my_arg',
        default_value='1234',
    )
    e_ac=ExecuteProcess(
```



```

        cmd=['echo', 'Value of my_arg:', launch_conf],
        output='screen',
    )

    return LaunchDescription([
        decl_my_arg,
        e_ac
    ])

```

| 5 urdf,xacro

| URDF

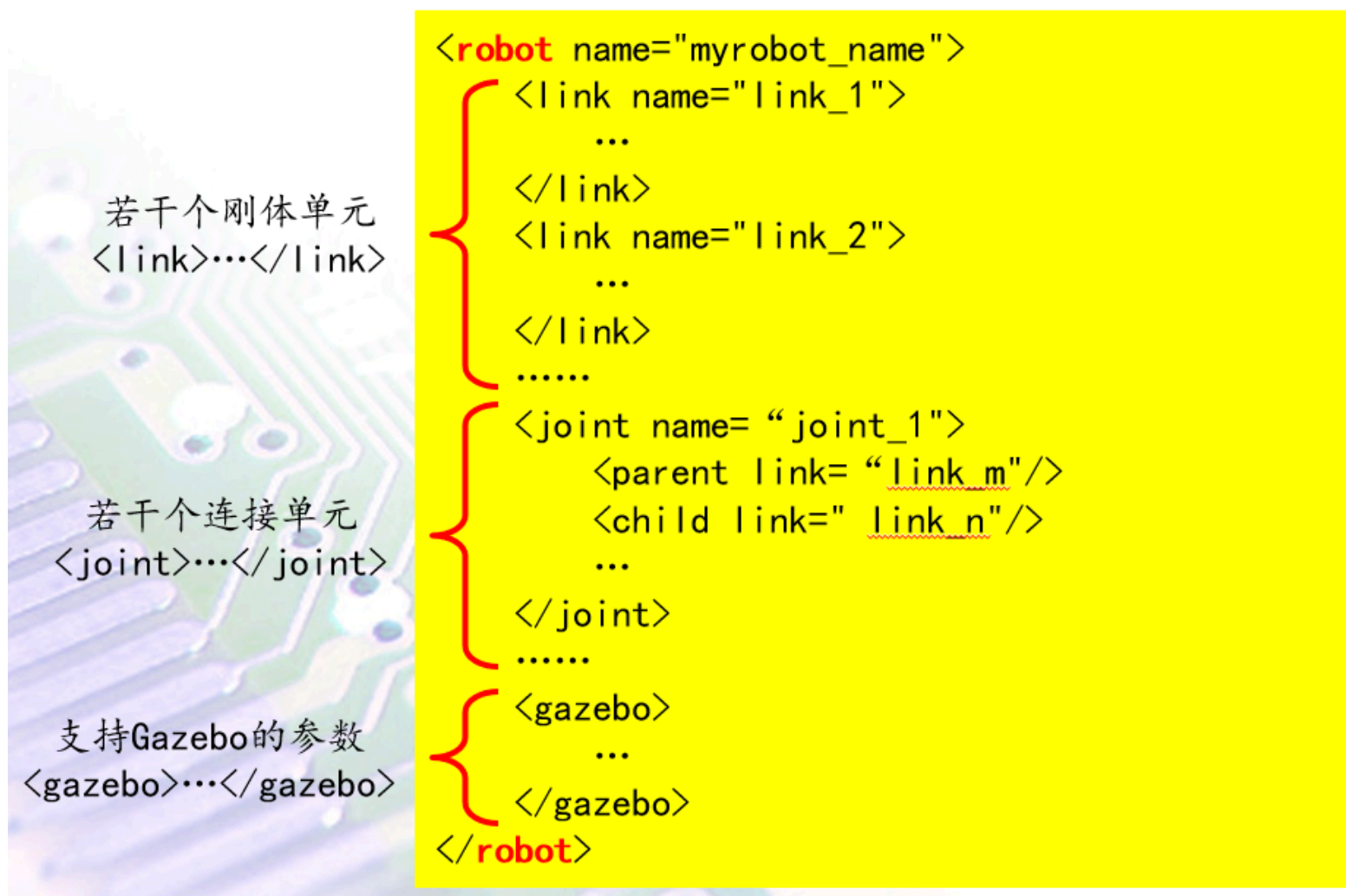
全称：统一机器人描述格式

| 概述

- 对机器人的刚体外观、物理属性等方面建模
- 结合动力学仿真器Gazebo，实现动力学仿真
- 描述语法：
 - 使用xml
 - 主要标签：
 - <robot>-顶层标签
 - <link>-刚体单元
 - <joint>-关节
 - <gazebo>-gazebo中的仿真参数

| 编程

- urdf文件编写
 - 格式：



The diagram illustrates the structure of a URDF file. On the left, there are three Chinese annotations with arrows pointing to specific parts of the XML code on the right:

- 若干个刚体单元 <link>...</link>** (Several rigid body units <link>...</link>): Points to the first two <link> tags.
- 若干个连接单元 <joint>...</joint>** (Several joint units <joint>...</joint>): Points to the <joint> tag.
- 支持Gazebo的参数 <gazebo>...</gazebo>** (Parameters supported by Gazebo <gazebo>...</gazebo>): Points to the <gazebo> tag.

The XML code on the right is as follows:

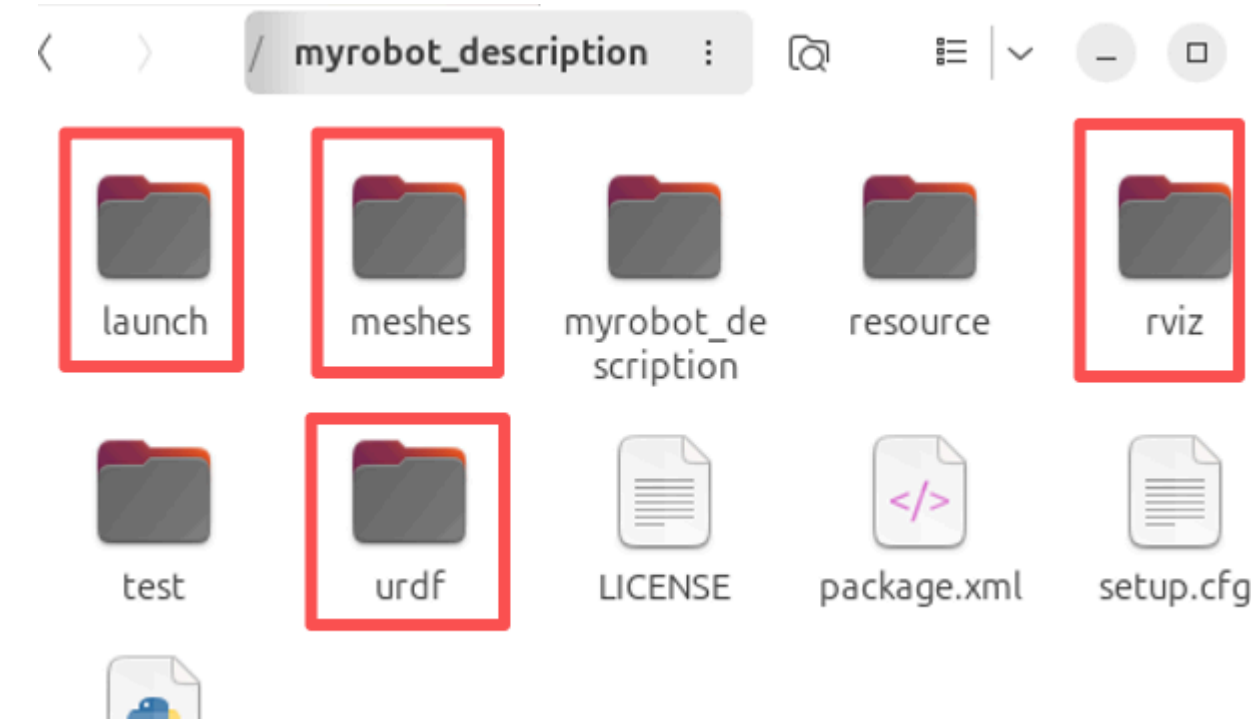
```

<robot name="myrobot_name">
  <link name="link_1">
    ...
  </link>
  <link name="link_2">
    ...
  </link>
  .....
  <joint name="joint_1">
    <parent link="link_m"/>
    <child link="link_n"/>
    ...
  </joint>
  .....
  <gazebo>
    ...
  </gazebo>
</robot>

```

- 详细内容见[机器人操作系统-副本/5 urdf,xacro > urdf文件编写](#)
- 创建功能包

• 创建子目录



- 编写launch.py
AI注释过的例程：

```
# 导入ROS 2启动系统所需的模块
from launch import LaunchDescription # 用于构建启动描述的主要类
from launch_ros.actions import Node # 用于创建ROS节点动作
from launch_ros.parameter_descriptions import ParameterValue # 参数值描述类
from ament_index_python import get_package_share_directory # 获取ROS 2功能包路径
from launch.substitutions import Command # 用于执行shell命令的替换类

# ROS 2启动文件的入口函数，必须命名为generate_launch_description
def generate_launch_description():
    # 【步骤1】获取机器人描述功能包的共享目录路径
    # 通过包名'myrobot_description'自动定位到该包的安装路径
    pkg_share_dir = get_package_share_directory('myrobot_description')

    # 【步骤2】拼接URDF模型文件的完整路径
    # 假设URDF文件位于包的urdf子目录下，文件名为myrobot.urdf
    urdf_file = pkg_share_dir + '/urdf/myrobot.urdf'

    # 【步骤3】处理URDF/Xacro文件
    # 使用xacro工具解析URDF文件（支持xacro宏和变量扩展）
    urdf_string = ParameterValue(
        # Command用于执行shell命令：'xacro 文件路径' 将xacro转换为URDF字符串
        Command(['xacro ', urdf_file]),
        value_type=str # 明确指定返回值为字符串类型
    )
    # 此时urdf_string包含了机器人模型的完整URDF描述（已解析的字符串形式）

    # 【步骤4】创建robot_state_publisher节点
    # 这是ROS 2中负责发布机器人状态的核心节点
    robot_state_publisher_node = Node(
        package='robot_state_publisher', # 节点所属的ROS包名
        executable='robot_state_publisher', # 要执行的可执行文件名
        parameters=[{ # 传递给节点的参数列表
            # 关键参数：将解析后的URDF模型字符串传递给节点
            'robot_description': urdf_string
        }]
    )
    # robot_state_publisher节点的作用：
    # 1. 从robot_description参数读取完整的URDF机器人模型
    # 2. 订阅/joint_states话题（获取各关节的实时状态）
    # 3. 发布/tf和/tf_static话题（计算并发布所有坐标系间的变换关系）
    # 4. 让RViz等可视化工具能够正确显示机器人模型及其运动

    # 【步骤5】构建并返回启动描述
    # 创建包含所有要启动节点的LaunchDescription对象
```

```

return LaunchDescription([
    robot_state_publisher_node # 将节点添加到启动列表中
])

```

- 修改setup.py
AI注释过的代码：

```

# 定义数据文件安装规则：将源文件复制到指定安装位置
data_files=[
    # 注册包到ROS 2资源索引
    ('share/ament_index/resource_index/packages', ['resource/' + package_name]),

    # 安装package.xml元数据文件
    ('share/' + package_name, ['package.xml']),
    # 安装launch目录下所有启动文件
    ('share/' + package_name + '/launch', glob('launch/*')),
    # 安装urdf目录下所有机器人模型文件
    ('share/' + package_name + '/urdf', glob('urdf/*')),
    # 安装rviz目录下所有可视化配置文件
    ('share/' + package_name + '/rviz', glob('rviz/*')),
    # 安装meshes目录下所有3D网格模型文件
    ('share/' + package_name + '/meshes', glob('meshes/*')),
],

```

- 为launch添加rviz节点

□ rviz可视化

□ 为launch.py增加rviz节点

```

rviz_config_file=pkg_share_dir + '/rviz/rviz_config.rviz'
rviz_node=Node(
    package="rviz2",
    executable="rviz2",
    name="my_test_bend_rviz",
    arguments=['-d',rviz_config_file],
)

return LaunchDescription([
    robot_state_publisher_node,
    rviz_node,
])

```

rviz配置
文件路径

设置固定坐标系
base_link

| urdf文件编写

| <robot>文件

```

<?xml version="1.0"?>
<robot name="edu_robot">
<!-- 这里包含定义这个机器人所有元素的代码 -->
</robot>

```

说明版本，机器人取名。

写了个包含各种要素的 AI 注释代码


```
<?xml version="1.0"?>
<robot name="simple_robot">

  <!-- 定义颜色材质 -->
  <material name="blue">
    <color rgba="0 0 0.8 1"/>  <!-- 蓝色 -->
  </material>
  <material name="red">
    <color rgba="0.8 0 0 1"/>  <!-- 红色 -->
  </material>

  <!-- 1. 底盘link - 基础圆柱体 -->
  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.1" radius="0.2"/>  <!-- 圆柱体 -->
      </geometry>
      <material name="blue"/>
    </visual>
  </link>
  <!-- 2. 身体link - 长方体，使用坐标偏移 -->
  <link name="body">
    <visual>
      <geometry>
        <box size="0.15 0.1 0.3"/>  <!-- 长方体 -->
      </geometry>
      <material name="red"/>
      <!-- 将身体向上移动，放在底盘上方 -->
      <origin xyz="0 0 0.2"/>
    </visual>
  </link>
  <!-- 3. 头部link - 球体 -->
  <link name="head">
    <visual>
      <geometry>
        <sphere radius="0.08"/>  <!-- 球体 -->
      </geometry>
      <!-- 没有指定材质，使用默认颜色 -->
    </visual>
  </link>

  <!-- 关节连接 -->
  <!-- 固定关节：身体固定在底盘上 -->
  <joint name="base_to_body" type="fixed">
    <parent link="base_link"/>
    <child link="body"/>
  </joint>
  <!-- 旋转关节：头部可以转动 -->
  <joint name="body_to_head" type="continuous">
    <parent link="body"/>
    <child link="head"/>
    <!-- 头部位于身体上方 -->
    <origin xyz="0 0 0.25"/>
    <!-- 绕Z轴旋转 -->
    <axis xyz="0 0 1"/>
  </joint>

</robot>
```

上面的link中还可补充碰撞属性<collision>，物理属性<inertial>

- <joint>关节类型

- continous-连续旋转
- prismatic-沿轴滑动
- revolute-有限旋转
- fixed-固定

| 检查urdf文件语法

```
check_urdf </path/to/file.urdf>
```

| XACRO

全称：可扩展标记语言(XML)宏(macro)

| 概述

优点：

- 更短小、更易读、便于复用代码（模块化）
 - 提供编程接口(常量、变量、数学公式或条件语句)，增强XML的表达力
- 特点：

- 需要xacro工具对.xacro进行解析

| 三大核心语法

| <xacro:property>常量

定义可重复用的，统一修改复制的常量值

例程：

```
<!-- 定义常量 -->
<xacro:property name="wheel_radius" value="0.1" />
<xacro:property name="body_length" value="0.5" />
<xacro:property name="pi" value="3.14159" />

<!-- 使用常量 -->
<link name="wheel">
  <visual>
    <geometry>
      <cylinder length="0.05" radius="${wheel_radius}" />
    </geometry>
  </visual>
</link>
```

| <xacro:macro>宏定义

将部件定义为可重复使用的宏，类似函数。

例程：

```
<!-- 定义一条机器腿的宏 -->
<xacro:macro name="robot_leg" params="prefix parent_link reflect">
  <link name="${prefix}_leg">
    <visual>
      <geometry><cylinder length="0.3" radius="0.05"/></geometry>
    </visual>
  </link>

  <joint name="${parent_link}_to_${prefix}_leg" type="fixed">
    <parent link="${parent_link}" />
    <child link="${prefix}_leg" />
  </joint>
</macro>
```

```
    <origin xyz="0 ${0.2*reflect} 0.15"/>
  </joint>
</xacro:macro>

<!-- 使用宏实例化两条腿 -->
<xacro:robot_leg prefix="left" parent_link="base_link" reflect="1"/>
<xacro:robot_leg prefix="right" parent_link="base_link" reflect="-1"/>
```

| <xacro:include>文件包含

将机器人的不同部分拆分到不同文件，有一个主文件负责组装

例程：

```
<!-- 主文件：robot.xacro -->
<?xml version="1.0"?>
<robot name="edu_robot" xmlns:xacro="http://www.ros.org/wiki/xacro">

  <!-- 包含其他组件 -->
  <xacro:include filename="$(find myrobot_description)/urdf/pebble_camera_usb.xacro"/>
  <xacro:include filename="$(find myrobot_description)/urdf/pebble_leg.xacro"/>
  <xacro:include filename="$(find myrobot_description)/urdf/pebble_gripper.xacro"/>

  <!-- 底盘基础结构 -->
  <link name="base_link">...</link>

  <!-- 实例化组件 -->
  <xacro:pebble_leg base_link_name="base_link" prefix="left" reflect="1"/>
  <xacro:camera_usb base_link_name="head" joint_rpy="0 0 0" joint_xyz="0.15 0 0.11"/>
</robot>
```

| 6 坐标变换TF

好像不是重点的样子.....

| 作业

在rviz2中显示urdf模型时，我们需要启动robot_state_publisher节点并设置其节点参数robot_description；对于活动关节还需要启动joint_state_publisher_gui节点，请阐述相关节点和话题以及参数的作用，说明其通信过程。



节点robot_state_publisher通过读取urdf文件内容，发布话题robot_description，用于rviz2显示模型，同时发布活动关节相关的/tf和固定关节相关的/tf_static话题；但是活动关节的/tf话题需要joint_state_publisher_gui提供/joint_state话题信息才得以完整，否则会导致活动关节无法正常连接从而导致相关link显示异常。

阅读第6章教学PPT中最后的escort_launch.py文件，阐述编程思想，列出重点使用到的类及其方法，说明这些方法在程序中的作用。

该launch程序涉及若干节点的启动，首先启动turtlesim_node节点在turtlesim窗口中生成乌龟turtle1，并通过turtle_teleop_key节点在独立控制台窗口实现键盘对turtle1的运动控制。在此之后通过调用3次escort_transform_publisher发布turtle1后、左、右3个方向的/tf_static话题，提供后续护卫乌龟追逐的坐标系id，并通过调用3次spawn函数生成3只后、左、右护卫乌龟。通过调用四次tf_broadcaster函数发布4只乌龟相对于世界坐标系的/tf话题。最后通过调用3次escort_transform_listener_and_twist_publisher函数计算turtle1旁（后、左、右）的某个护卫坐标id相对于某个护卫乌龟id的变换关系，并将该关系转换为控制护卫乌龟运动的话题进行发布。

以上步骤的各个函数应用了若干和TF相关的重要的类，包括：

(1) static_transform_publisher：

能够发布两坐标系之间的静态变换关系/tf_static，即实现了turtle1后、左、右三处坐标id和turtle1自身坐标id的静态变换关系。

(2) TransformBroadcaster：

其中的sendTransform方法能够发布两坐标系的动态变换关系/tf，即实时发布乌龟坐标和世界坐标之间的变换关系/tf。

(3) TransformListener:

通过内嵌的缓存对象的lookup_transform方法计算两坐标系id之间的变换关系，即通过计算护卫坐标id到护卫乌龟id之间的变换关系，以便计算生成/cmd_vel话题控制护卫乌龟运动。

备注，为了避免给同学们带来困扰，以上没有用“目标坐标id”这个词语，因为在此应用中，lookup_transform函数中的参数名“target_frame”取值是护卫乌龟的id，并不是乌龟要运动到的目的地坐标id。